Assignment-4

Data Structures

Instructions:

- In case of any kind of plagiarism or dishonesty, Instructors won't be hesitant to give 'F' grade.
- Deadline is final & won't be extended in any case.
- Start assignment right now. You won't be able to complete it on last 2 days :)
- You have to submit two.cpp files named as "sender.cpp" and "reciever.cpp"
- The use of any external libraries such as String, cmath etc. is NOT allowed.
- Late submission will result in zero marks in the assignment. Try to submit the assignment at least one day before the deadline to avoid any issues at the last moment.
- Exception Handling and Menu based implementation is part of Assignment
- You have to add both your solution (.cpp) files in a single folder, compress that folder into a ***.zip*** file and submit that file. Please name the zip file after your roll no (A3_23I-XXXX.zip).

Question 1

Description:

In this assignment, you will create a sender program and a receiver program for efficient communication between users.

Sender Program:

Imagine you have a piece of text stored in a file. The sender program's job is to read this text and shrink its size by encoding it. Encoding here means assigning shorter codes (bits) to the characters that appear more frequently in the text. For example, if the letter 'A' appears a lot in the text, then rather than storing its ASCII value of "1000001" it might be represented by a shorter code such as "00" or "101" etc. The program will then save this smaller, encoded data into a new file.

Receiver Program:

The receiver program does the opposite. It reads the encoded data from the file created by the sender program. Its task is to transform the output of sender program back to the original text data. Essentially, it reverses the encoding process done by the sender program and shows the original text to the user.
In a nutshell, the sender makes the data smaller by representing it character by less number of bits, and the receiver takes this encoded data and turns it back into the original text so that people can understand the message. This efficient method of communication is essential in various fields like computer science and telecommunications.

Let's delve into the steps of the sender process:

Step 1: Take name of a text file and read its content

Step 2: Determine the frequency of each character.

Step 3: Create separate node for each character based on its frequency. Each node will store a character, its frequency, address of left child, and address of right child. The left and right pointers will be NULL, if a node doesn't have any child.

Step 4: Merge the available nodes iteratively.

1. Pick the two nodes having minimum frequency values
2. Calculate the sum of both the frequencies
3. Store the sum value in a new node
4. Make the two selected nodes as children of this new node. Make the least frequency character as a left child of new node and make second least frequency character as a right child 5. Exclude the left and right nodes from the merging process
6. The new node can be selected for merging now.

Step 5: Continue merging nodes until we are left with only one node.

Step 6: Assign binary codes to characters based on their position in the tree, ensuring shorter codes for more common characters.

Step 7: Encode the text of file using binary codes and store the output in a new text file (output.txt).

Step 8: Store the codes in a text file (codes.txt)

Here is an example for your clarity:

Example:

Step 1: Let's consider the text: "AABBAACDDD".

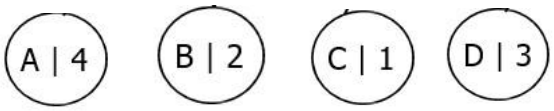
Step 2: Calculate the frequency of each character in the text:

Character	**Frequency**
A	4
B	2
C	1
D	3

Step 3: Create nodes for each character based on their frequency:

Nodes:

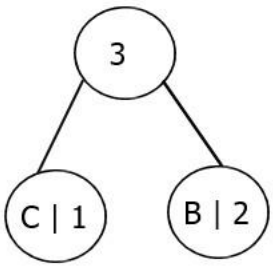
A(4), B(2), C(1), D(3)



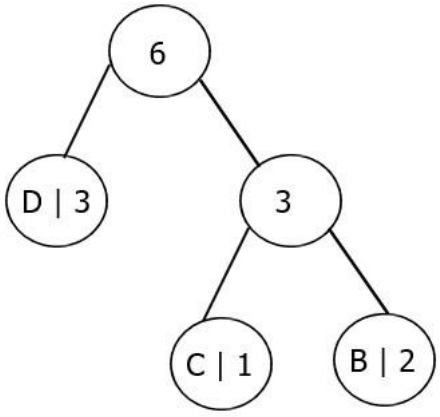
Step 4 & 5: Merge nodes iteratively:

Using this algorithm, the characters can be encoded as follows:

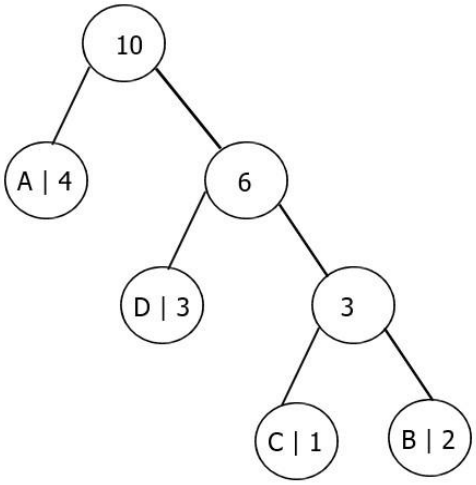
1. Merge C and B: CB (Frequency: 3)



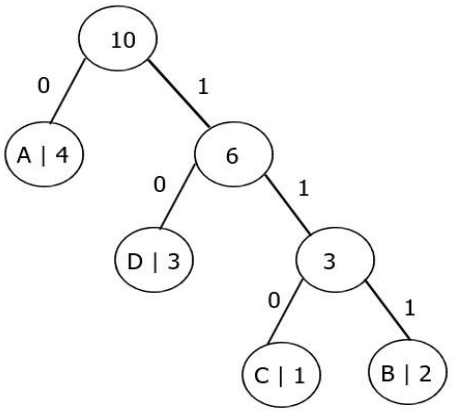
2. Merge CB and D: CBD (Frequency: 6)



3. Merge A and CBD: ACBD (Frequency: 10)



Step 6:
Assign binary codes to characters based on their position in the tree. Assign ‘0’ to left child and ‘1’ to right child on every node.



Codes are as follows:

- A: 0
- D: 10
- C: 110
- B: 111

Step 7: Encode the text of file using binary codes.

AABBAACDDD ->	00111111001101010 10

Store the binary encoded information in “output.txt”

Now, let's delve into the steps of the **receiver** program:

Step 1: Read the “encoded.txt” and “codes.txt”

Step 2: Iterate over encoded text, and find the character against each code”.

Code	0	0	111	111	0	0	110	10	10	10
Letter	A	A	B	B	A	A	C	D	D	D

Step 3: Save the decoded text in “decoded.txt”, and also show it on screen.

Question 2
Adaptive Indexing and Consistency Engine over Large-Scale Transaction Data

Dataset Requirement

Dataset: IEEE-CIS Fraud Detection Dataset
Link: https://www.kaggle.com/competitions/ieee-fraud-detection/data

Use a subset (e.g., first 50,000 rows). Data must be read directly from CSV.
Problem Statement

You are required to design an adaptive indexing engine that processes real transaction data from a CSV file and maintains three synchronized hierarchical structures:

1. A Binary Tree representing raw ingestion order
2. A Binary Search Tree indexing transactions by Transaction Amount
3. An AVL Tree indexing transactions by Risk Probability (or a derived fraud score)

The system must dynamically maintain consistency across all structures while supporting selective rollback, partial reconstruction, and anomaly-based restructuring.

Data Mapping

From dataset, extract:

transactionID → TransactionID
amount → TransactionAmt

timestamp → TransactionDT
riskScore → derived (e.g., normalized TransactionAmt or custom heuristic)

Core Requirements

Streaming File Ingestion

Read the dataset line by line from CSV.

Each record must:

- Be inserted into a Binary Tree using level-order logic
- Simultaneously inserted into BST (based on amount)
- Simultaneously inserted into AVL Tree (based on riskScore)

Constraint:

No bulk loading.
Insertion must be incremental and synchronized.

Adaptive Risk Recalibration

At runtime, the system may receive:

RECALIBRATE α

This changes the riskScore formula dynamically.

After recalibration:

- AVL Tree must update itself WITHOUT full reconstruction
- Only affected nodes should be repositioned
- BST and Binary Tree must remain structurally valid

This requires selective deletion + reinsertion.

Disk-Based Partial Reload

The system must support:

RELOAD k

Meaning:

- Remove last k inserted transactions from memory structures
- Reload them again from the CSV file
- Ensure final structures remain identical to original insertion

Constraint:

File pointer management must be handled manually.

Anomaly Window Extraction

For a given threshold X:

• Extract all transactions where amount ≥ X
• These must form a new BST

Then:

• Convert that BST into an AVL Tree with minimum rotations

Constraint:

No sorting arrays
Must use tree transformations only

Structural Drift Detection

Define drift as:

A node whose position differs significantly between BST and AVL (based on depth difference > D)

The system must:

• Detect all drift nodes
• Rebalance only affected subtrees in AVL
• Adjust BST accordingly without violating ordering

Temporal Range Isolation

Given time range [T1, T2]:

• Extract corresponding nodes from all structures
• Form a separate balanced AVL Tree

Constraint:

Must not duplicate entire nodes in memory
Use pointer/reference-based extraction

Multi-Level Consistency Verification

At any point, system must verify:

• In-order traversal of BST is sorted
• AVL Tree is height-balanced
• Binary Tree preserves insertion order

All checks must be done in a single combined traversal strategy.

Memory-Constrained Rebalancing

If memory usage exceeds limit M:

• Compress BST into a minimal-height tree
• Reconstruct AVL Tree from compressed BST

Constraint:

No array flattening
Must use tree rotations and pointer restructuring

Persistent Checkpointing

System must periodically:

• Save structure snapshots to file
• Reload them later

Constraint:

• Must serialize tree structures manually
• No built-in serialization libraries

Deterministic Reconstruction Guarantee

After any sequence of:

• Recalibration
• Reload
• Drift correction
• Memory compression

Re-running the program on the same dataset must produce identical structures.

Constraints

No STL
No recursion for core ops
No arrays for full dataset
Manual AVL rotations
File handling required

Grading Rubric (100 Marks)

Component	Description	Marks
File Handling	CSV reading, reload	10
Binary Tree	Level-order correctness	10
BST	Ordering and ops	10
AVL Tree	Balancing and rotations	15
Synchronization	Across structures	15
Advanced Logic	Recalibration, drift	15
Analysis	Efficiency	10
Checkpointing	Save/load	5
Determinism	Reproducibility	5

Code Quality	Structure and safety	5

Good Luck 😊